

Chapter 5. Deep Deterministic Policy Gradient

Samsung AI Expert
Reinforcement Learning

Jul 9, 2025

DDPG - Review

Recap : DQN aims to learn Q , and choose action greedily as follows:

$$a_t = \arg \max_a Q(s_t, a; \theta),$$

Now, instead of choosing action a_t directly by solving

$$\max_a Q(s_t, a; \theta),$$

we employ a separate **actor network** π_ϕ and just select a_t by

$$a_t = \pi_\phi(s_t).$$

→ **Actor-Critic methods**

DDPG - Review

Updating θ ?

Training $Q(s, a; \theta)$ in DDPG is simple as same as training it in DQN:

$$\text{DQN} : y_j = r_j + \gamma \max_a Q(s'_j, a; \theta) \quad (\text{TD target})$$

$$\text{DDPG} : y_j = r_j + \gamma Q(s'_j, \pi_\phi(s_j); \theta)$$

We solve (by performing gradient descent)

$$\min_{\theta} \frac{1}{|B|} \sum_j |Q(s_j, a_j; \theta) - y_j|^2.$$

not $\pi_\phi(s_j)$!

DDPG - Review

How to tune params ϕ of the actor π_ϕ ?

Our original goal was to solve

$$\max_a Q(s_t, a; \theta),$$

so we expect π_ϕ to satisfy

$$Q(s_t, \pi_\phi(s_t); \theta) \approx \max_a Q(s_t, a; \theta)$$

This naturally leads to the following optimization problem!

$$\max_{\phi} Q(s_t, \pi_\phi(s_t); \theta)$$

TODO_1)



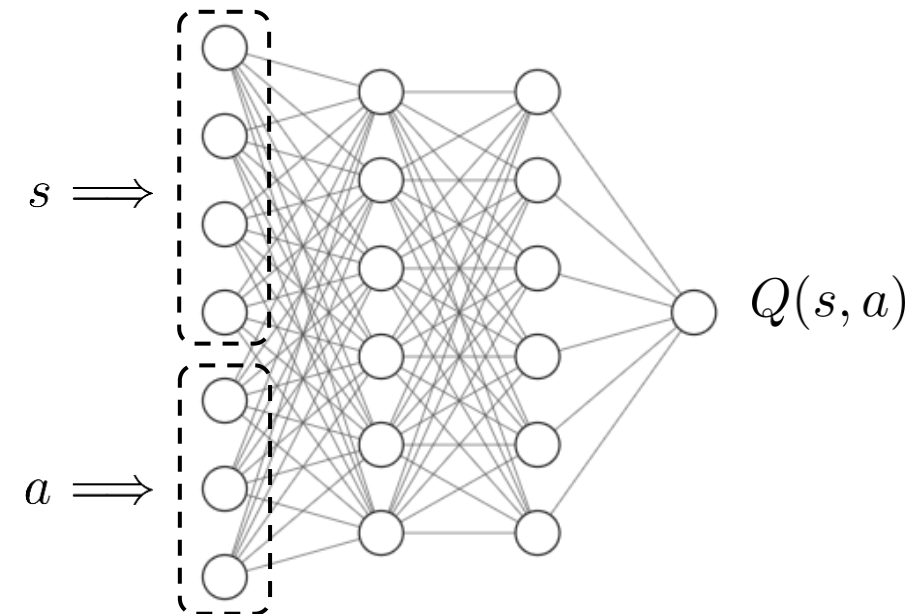
TODO_1)

```
return self.ctrl_range * torch.tanh(self.fc3(x))
```

DDPG - Implementation

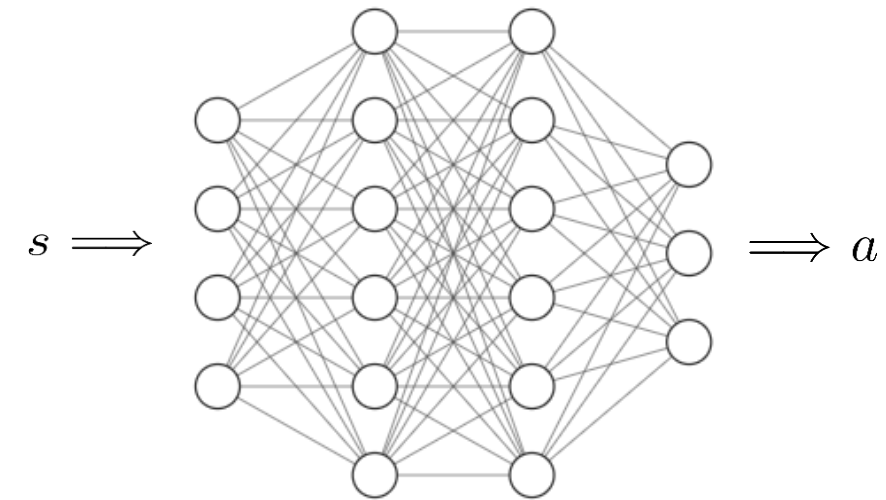
```
1  class Critic(nn.Module):
2      def __init__(self, obs_dim, act_dim, hidden1, hidden2):
3          super(Critic, self).__init__()
4          self.fc1 = nn.Linear(obs_dim + act_dim, hidden1)
5          self.fc2 = nn.Linear(hidden1, hidden2)
6          self.fc3 = nn.Linear(hidden2, 1)
7
8      def forward(self, obs, act):
9          x = torch.cat([obs, act], dim=1)
10         x = F.relu(self.fc1(x))
11         x = F.relu(self.fc2(x))
12         return self.fc3(x)
```

feed state-action pair



DDPG - Implementation

```
1  class Actor(nn.Module):
2      def __init__(self, obs_dim, act_dim, ctrl_range, hidden1, hidden2):
3          super(Actor, self).__init__()
4          self.fc1 = nn.Linear(obs_dim, hidden1)
5          self.fc2 = nn.Linear(hidden1, hidden2)
6          self.fc3 = nn.Linear(hidden2, act_dim)
7          self.ctrl_range = ctrl_range
8
9      def forward(self, obs):
10         x = F.relu(self.fc1(obs))
11         x = F.relu(self.fc2(x))
12         return self.ctrl_range * torch.tanh(self.fc3(x))
```



for control constraints $-\bar{u} \leq u \leq \bar{u}$ (physical limitation)

TODO_2)

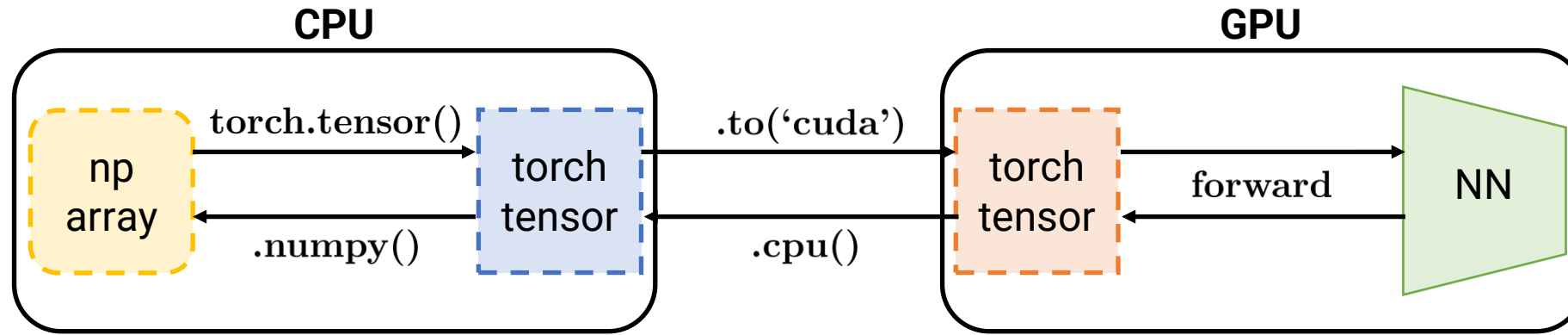


TODO_2)

```
act_tensor = self.actor(obs_tensor)
```

DDPG - Implementation

How to compute an action/control?



```
1 def act(self, obs):
2     obs = obs[np.newaxis, ...]
3     with torch.no_grad():
4         obs_tensor = torch.Tensor(obs).to(device)
5         act_tensor = self.actor(obs_tensor)
6     action = act_tensor.cpu().detach().numpy()
7     action = np.squeeze(action, axis=0)
8     return action
```

main part!

TODO_3)

TODO_4)



TODO_3)

```
target = rew + gamma * mask * target_critic(next_obs, target_actor(next_obs))
```

TODO_4)

```
critic_loss = torch.mean((out - target) ** 2)
```

DDPG - Implementation

```
target = rewards + self.gamma * mask * self.targ_Q(next_observations, self.targ_pi(next_observations))
```

```
out = self.Q(observations, actions)
loss_ftn = MSELoss()
loss = loss_ftn(out, target)
self.Q_optimizer.zero_grad()
loss.backward()
self.Q_optimizer.step()
```

$Q(s, a; \theta)$

This is how we train DDPG!

```
pi_loss = - torch.mean(self.Q(observations, self.pi(observations)))
self.pi_optimizer.zero_grad()
pi_loss.backward()
self.pi_optimizer.step()
```

$\pi_{\phi}(s)$

Tip. freeze networks properly

DDPG - Implementation

1. critic network training : what's different from DQN?

```
1  def update(agent, replay_buf, gamma, actor_optim, critic_optim, target_actor, target_critic, tau, batch_size):
2
3      batch = replay_buf.sample_batch(batch_size=batch_size)
4
5      with torch.no_grad():
6          obs = torch.Tensor(batch.obs).to(device)
7          act = torch.Tensor(batch.act).to(device)
8          next_obs = torch.Tensor(batch.next_obs).to(device)
9          rew = torch.Tensor(batch.rew).to(device)
10         done = torch.Tensor(batch.done).to(device)
11         mask = 1. - done
12
13         target = rew + gamma * mask * target_critic(next_obs, target_actor(next_obs))
14     out = agent.critic(obs, act)
15
16     loss_ftn = MSELoss()
17     critic_loss = loss_ftn(out, target)
18     critic_optim.zero_grad()
19     critic_loss.backward()
20     critic_optim.step()
```

$Q(s', \pi(s'))$ instead of $\max_{a'} Q(s', a')$!

TODO_5)



Learning and Decision Systems

TODO_5)

```
actor_loss = -torch.mean(agent.critic(obs, agent.actor(obs)))
```

DDPG - Implementation

2. actor network training

```
1 actor_loss = -torch.mean(agent.critic(obs, agent.actor(obs)))
2
3 actor_optim.zero_grad()
4 actor_loss.backward()
5 actor_optim.step()
```

This tries to solve
 $\max_{\phi} Q(s_t, \pi_{\phi}(s_t); \theta).$

TODO_7)

TODO_7)

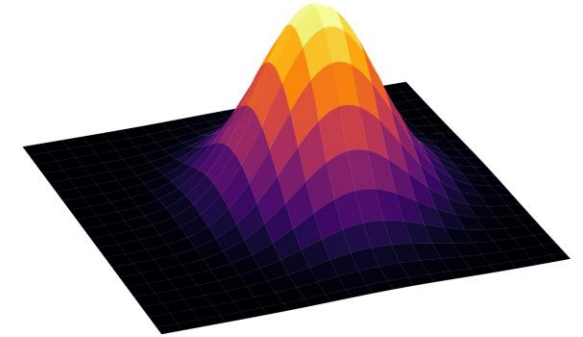
```
action = agent.act(obs) + noise_std * np.random.randn(act_dim)
```

DDPG - Implementation

Putting these together, we get...

```
1  obs = env.reset()
2  done = False
3  step_count = 0
4
5  for t in range(num_updates):
6      action = agent.act(obs) + noise_std * np.random.randn(act_dim)
7      action = np.clip(action, -ctrl_range, ctrl_range)
8
9      next_obs, rew, done, _ = env.step(action)
10     replay_buf.append(obs, action, next_obs, rew, done)
11     obs = next_obs
12     step_count += 1
13
14     if step_count == ep_len:
15         obs = env.reset()
16         done = False
17         step_count = 0
18
19     if t % train_interval == 0:
20         for _ in range(train_interval):
21             update(...)
```

Why some random noise?
⇒ For exploration! (cf. ϵ -greedy)



reset the episode

Thank you!



Learning and Decision Systems